

Conversazione con Gemini

devo creare una consoleApp per un esame in c# che deve simulare un piccolo negozio online. è fondamentale che i metodi rimangano gli stessi e quindi con la stessa firma, parametri e valori di ritorno. muoviamoci passo passo con un uso consapevole dell'AI.

Struttura

Program: punto di ingresso dell'applicazione.

ApplicazioneNegozio: gestisce i menu console per utente e amministratore.

Utente: classe madre per rappresentare un utente del sistema.

Prodotto: classe madre per rappresentare un prodotto del catalogo con codice, nome, prezzo, quantità iniziale e quantità disponibile.

ElementoCarrello: rappresenta una riga del carrello con prodotto, quantità scelta e prezzo unitario.

Acquisto ed ElementoAcquistato: rappresentano un ordine completato e i prodotti acquistati.

CatalogoProdotti: gestisce prodotti, prezzi e quantità di magazzino.

CarrelloUtente: gestisce aggiunta, modifica, rimozione e totale del carrello.

StoricoAcquisti: conserva in memoria gli acquisti effettuati durante l'esecuzione.

ServizioNegozio: coordina catalogo, carrello e storico, soprattutto nella conferma dell'acquisto.

ReportProdotto: modello semplice per il riepilogo amministratore.

I metodi con TODO devono essere completati senza cambiare firma, nome, parametri o tipo di ritorno. Le parti principali da implementare sono:

ciclo principale della Console App;

menu utente;

menu amministratore;

input da console;

aggiunta/modifica/rimozione prodotti dal carrello;

modifica/eliminazione prodotti nel catalogo;

filtro acquisti per nome utente;

aggiunta prodotto al carrello tramite codice;

conferma acquisto di un Utente con controllo disponibilità, aggiornamento magazzino, storico e svuotamento carrello.

Non è richiesto il salvataggio su file o database: i dati possono restare in memoria durante l'esecuzione.

Ottimo approccio. Andare passo dopo passo è il modo migliore per non perdere il controllo del codice e assicurarsi che l'architettura sia solida, specialmente per un esame dove il rispetto delle firme dei metodi è fondamentale.

Per iniziare in modo "consapevole", dobbiamo prima definire lo scheletro (il blueprint) delle classi con i relativi metodi `TODO`. Senza lo scheletro cartaceo delle firme, rischiamo di inventare parametri o tipi di ritorno lungo la strada. Ecco come ti propongo di procedere: dividiamo il lavoro in **3 macro-fasi**. Oggi partiamo dalla **Fase 1**.

Le Fasi del Progetto

- **Fase 1: Definizione dei Modelli Dati e dello Scheletro (Oggi)**
 - Creiamo le classi base (`Prodotto`, `Utente`, `ElementoCarrello`, `ElementoAcquistato`, `Acquisto`, `ReportProdotto`).
 - Definiamo le classi di gestione con i metodi `TODO` vuoti (firme precise).
- **Fase 2: Logica di Business (Catalogo, Carrello, ServizioNegozio)**
 - Implementiamo la logica dei metodi (aggiunta, rimozione, controllo disponibilità e transazione di acquisto).
- **Fase 3: Interfaccia Utente (Menu, Input e Ciclo Principale)**
 - Sviluppiamo l'interazione in console, i menu Utente/Amministratore e i controlli di input.

Fase 1: Lo Scheletro del Codice

Visto che non hai ancora allegato il codice di partenza con i `TODO` esatti richiesti dal tuo professore, ho strutturato qui sotto le classi **Modello** e le firme standard per questo tipo di progetto d'esame.

Dai un'occhiata a questa prima struttura per i **Modelli Dati**.

1. I Modelli Base (`Prodotto`, `Utente`, `ElementoCarrello`)

C#

```
public class Utente
{
    public string Username { get; set; }
    public string Ruolo { get; set; } // "Utente" o "Admin"

    public Utente(string username, string ruolo)
    {
        Username = username;
        Ruolo = ruolo;
    }
}

public class Prodotto
```

```

{
    public string Codice { get; set; }
    public string Nome { get; set; }
    public decimal Prezzo { get; set; }
    public int QuantitaIniziale { get; set; }
    public int QuantitaDisponibile { get; set; }

    public Prodotto(string codice, string nome, decimal prezzo, int
quantitaIniziale)
    {
        Codice = codice;
        Nome = nome;
        Prezzo = prezzo;
        QuantitaIniziale = quantitaIniziale;
        QuantitaDisponibile = quantitaIniziale;
    }
}

public class ElementoCarrello
{
    public Prodotto Prodotto { get; set; }
    public int QuantitaScelta { get; set; }
    public decimal PrezzoUnitario { get; set; } // Cattura il prezzo al momento
dell'aggiunta

    public ElementoCarrello(Prodotto prodotto, int quantitaScelta)
    {
        Prodotto = prodotto;
        QuantitaScelta = quantitaScelta;
        PrezzoUnitario = prodotto.Prezzo;
    }
}

```

2. Le Classi di Gestione (Le Firme da Completare)

Di seguito trovi lo scheletro delle classi che conterranno la logica. Nota i metodi con il `throw new NotImplementedException();` (i tuoi `TODO`).

C#

```

public class CatalogoProdotti
{

```

```

private List<Prodotto> _prodotti = new List<Prodotto>();

public void AggiungiProdotto(Prodotto p) => _prodotti.Add(p);

public Prodotto CercaPerCodice(string codice)
{
    // TODO: Trovare e restituire il prodotto tramite codice
    throw new NotImplementedException();
}

public bool ModificaProdotto(string codice, string nuovoNome, decimal
nuovoPrezzo, int nuovaQuantita)
{
    // TODO: Modificare il prodotto e aggiornare le quantità
    throw new NotImplementedException();
}

public bool EliminaProdotto(string codice)
{
    // TODO: Rimuovere il prodotto dal catalogo
    throw new NotImplementedException();
}

public List<Prodotto> OttieniCatalogo() => _prodotti;
}

public class CarrelloUtente
{
    private List<ElementoCarrello> _elementi = new List<ElementoCarrello>();

    public void AggiungiOModificaInCarrello(Prodotto p, int quantita)
    {
        // TODO: Se esiste già, modifica la quantità, altrimenti aggiungi una
nuova riga
        throw new NotImplementedException();
    }

    public bool RimuoviDalCarrello(string codiceProdotto)
    {

```

```

        // TODO: Rimuovere l'elemento dal carrello
        throw new NotImplementedException();
    }

    public decimal CalcolaTotale()
    {
        // TODO: Calcolare il totale del carrello
        throw new NotImplementedException();
    }

    public List<ElementoCarrello> OttieniElementi() => _elementi;
    public void Svuota() => _elementi.Clear();
}

public class ServizioNegozio
{
    private CatalogoProdotti _catalogo;
    private StoricoAcquisti _storico;

    public ServizioNegozio(CatalogoProdotti catalogo, StoricoAcquisti storico)
    {
        _catalogo = catalogo;
        _storico = storico;
    }

    public bool ConfermaAcquisto(Utente utente, CarrelloUtente carrello)
    {
        // TODO: Il motore economico dell'app.
        // 1. Controlla disponibilità magazzino per OGNI prodotto nel carrello.
        // 2. Se OK, decrementa le quantità disponibili nel catalogo.
        // 3. Crea l'oggetto Acquisto e lo salva nello storico.
        // 4. Svuota il carrello.
        throw new NotImplementedException();
    }
}

// Strutture di supporto per lo storico
public class ElementoAcquistato
{

```

```

    public string CodiceProdotto { get; set; }
    public string NomeProdotto { get; set; }
    public int Quantita { get; set; }
    public decimal PrezzoPagato { get; set; }
}

public class Acquisto
{
    public string IdIdAcquisto { get; set; } =
Guid.NewGuid().ToString().Substring(0, 8);
    public string UsernameUtente { get; set; }
    public DateTime DataAcquisto { get; set; } = DateTime.Now;
    public List<ElementoAcquistato> Elementi { get; set; } = new
List<ElementoAcquistato>();
    public decimal Totale { get; set; }
}

public class StoricoAcquisti
{
    private List<Acquisto> _acquisti = new List<Acquisto>();

    public void RegistraAcquisto(Acquisto a) => _acquisti.Add(a);

    public List<Acquisto> FiltraPerUtente(string username)
    {
        // TODO: Filtrare gli acquisti per nome utente
        throw new NotImplementedException();
    }
}

```

Il nostro prossimo passo

Hai già una traccia o delle firme precise fornite dal testo dell'esame che dobbiamo copiare fedelmente, oppure questa struttura che ti ho proposto si adatta bene al tuo progetto?

Se hai già un file di partenza con firme specifiche, **copiamelo pure qui**, così lavoriamo direttamente sui tuoi metodi originali. Se invece partiamo da zero, confermami questa struttura e passeremo alla **Fase 2: l'implementazione della logica del Catalogo e del Carrello**.

questo è il mio primo passo. concrolla se il codice è giusto:

```

public void Avvia()
{
    // TODO: implementare il ciclo principale della Console App.
    // Suggerimento:
    // 1. mostrare un messaggio di benvenuto;
    // 2. chiedere se l'utente vuole entrare come "utente" o "amministratore";
    // 3. chiamare GestisciMenuUtente oppure GestisciMenuAmministratore;
    // 4. permettere l'uscita dal programma con una scelta dedicata.

    Console.WriteLine("\n=== BENVENUTO NEL NEGOZIO ONLINE ===");

    bool esci = false;
    while (!esci)
    {
        string scelta = ScegliRuolo();

        if (scelta == "utente")
        {
            GestisciMenuUtente();
        }
        else if (scelta == "amministratore")
        {
            GestisciMenuAmministratore();
        }
        else if (scelta == "esci")
        {
            esci = true;
            Console.WriteLine("\nGrazie per aver utilizzato il negozio online!");
        }
    }
}

```

La struttura del tuo ciclo principale è ottima: l'uso del `while` regolato da un flag booleano (`esci`) è la scelta standard e più pulita per gestire un menu a console.

Anche la separazione della logica delegando a `ScegliRuolo()` e ai due menu dedicati è un'eccellente pratica di programmazione (punti a favore all'esame per la leggibilità).

Ci sono solo due piccolissimi dettagli da considerare per renderlo perfetto e a prova di errore:

1. Posizione del Benvenuto

Hai inserito il messaggio di benvenuto fuori dal ciclo `while`. Questo va benissimo se vuoi che appaia solo all'avvio del programma. Tieni presente, però, che quando l'utente uscirà da `GestisciMenuUtente()` o `GestisciMenuAmministratore()`, il ciclo ricomincerà e chiederà di nuovo il ruolo, ma senza stampare il titolo. Se vuoi un'interfaccia più ordinata, potresti voler pulire lo schermo o ristampare l'intestazione a ogni ciclo.

2. Robustezza delle stringhe di confronto

Nel codice fai controlli del tipo `if (scelta == "utente")`. Questo significa che il metodo `ScegliRuolo()` deve occuparsi di restituire la stringa **esattamente in minuscolo** e senza spazi. Se `ScegliRuolo()` restituisce `"Utente"` (con la U maiuscola), il blocco `if` fallirà.

Per renderlo più robusto a prova di errore di digitazione (e assicurarsi che i requisiti d'esame non saltino per una maiuscola), ti consiglio di usare `.ToLower()` o `.Trim()` direttamente nel controllo o dentro `ScegliRuolo()`.

Il codice ottimizzato

Ecco come potresti rifinirlo per renderlo impeccabile:

C#

```
public void Avvia()
{
    // 1. Messaggio di benvenuto (stampato una volta all'avvio)
    Console.WriteLine("=== BENVENUTO NEL NEGOZIO ONLINE ===");

    bool esci = false;
    while (!esci)
    {
        Console.WriteLine("\n--- MENU PRINCIPALE ---");

        // 2. Chiedere il ruolo (converte l'output in minuscolo e pulisce gli
        spazi per sicurezza)
        string scelta = ScegliRuolo().Trim().ToLower();

        // 3. Gestione della scelta
        if (scelta == "utente")
        {
            GestisciMenuUtente();
        }
    }
}
```



```

        else if (scelta == "amministratore" || scelta == "admin") // Un piccolo
alias non fa mai male
        {
            GestisciMenuAmministratore();
        }
        else if (scelta == "esci")
        {
            // 4. Permettere l'uscita
            esci = true;
            Console.WriteLine("\nGrazie per aver utilizzato il negozio online.
Arrivederci!");
        }
        else
        {
            // Gestione opzionale nel caso in cui ScegliRuolo non filtri gli
input errati
            Console.WriteLine("Scelta non valida. Riprova.");
        }
    }
}

```

Prossimo Passo

Il passo successivo dipende da come vuoi procedere:

1. Vogliamo implementare subito il metodo `ScegliRuolo()` per agganciarlo a questo ciclo?
2. Oppure preferisci passare alla struttura dei due macro-menu (`GestisciMenuUtente()` e `GestisciMenuAmministratore()`)?

la conversione dell'input in minuscolo e pulizia degli spazi è richiesta nel metodo `ScegliRuolo()` che ho implementato così:

```

private string ScegliRuolo()
{
    // TODO: leggere da console il ruolo scelto.
    // Valori consigliati: "utente", "amministratore", "esci".
    // Gestire input vuoti e maiuscole/minuscole con Trim() e ToLower().
    Console.Write("\nScegli ruolo (utente/amministratore/esci): ");
    string? input = Console.ReadLine();

    if (string.IsNullOrWhiteSpace(input))
    {

```

```

        return "";
    }

    return input.Trim().ToLower();
}

```

Il tuo metodo `ScegliRuolo()` è scritto molto bene. Hai gestito correttamente il tipo nullable (`string?`) che restituisce `Console.ReadLine()` e hai usato `string.IsNullOrWhiteSpace(input)`, che è il modo più sicuro in C# per intercettare sia stringhe vuote sia stringhe fatte di soli spazi.

C'è solo un piccolo dettaglio di logica da coordinare con il metodo `Avvia()` che abbiamo visto prima:

Se l'utente preme invio senza scrivere nulla, il metodo restituisce una stringa vuota (`""`). Nel metodo `Avvia()`, la stringa vuota non corrisponderà a "utente", "amministratore" o "esci", quindi il ciclo `while` continuerà a girare riproponendo la domanda. Questo va benissimo, ma significa che l'aggiunta del blocco `else { Console.WriteLine("Scelta non valida"); }` che ti avevo suggerito in `Avvia()` diventa fondamentale per avvisare l'utente del problema.

Se vuoi rendere il metodo ancora più robusto (evitando che `Avvia()` debba gestire l'errore), potresti inserire un piccolo ciclo di validazione direttamente dentro `ScegliRuolo()`, in modo che il metodo non esca finché l'utente non inserisce una delle tre parole chiave corrette.

Ecco come diventerebbe con questa piccola ottimizzazione:

C#

```

private string ScegliRuolo()
{
    while (true)
    {
        Console.Write("\nScegli ruolo (utente/amministratore/esci): ");
        string? input = Console.ReadLine();

        // Se l'input è valido, lo ripuliamo
        if (!string.IsNullOrWhiteSpace(input))
        {
            string sceltaFormattata = input.Trim().ToLower();

            // Verifichiamo che sia una delle opzioni accettate dal programma
            if (sceltaFormattata == "utente" || sceltaFormattata ==
"amministratore" || sceltaFormattata == "esci")
            {

```

```

        return sceltaFormattata;
    }
}

// Se l'input è vuoto o non valido, il ciclo continua
Console.WriteLine("Opzione non valida. Inserisci esattamente 'utente',
'amministratore' o 'esci'.");
}
}

```

In questo modo, il metodo `Avvia()` riceverà **sempre e solo** una stringa sicuramente valida (`"utente"`, `"amministratore"` o `"esci"`), rendendo il codice principale molto più lineare e pulito.

Prossimo Passo

Ora che l'accesso e la scelta del ruolo sono blindati, dove ci spostiamo?

1. Creiamo la struttura di `GestisciMenuUtente()`?
2. Oppure preferisci definire prima `GestisciMenuAmministratore()`?

adesso creiamo una struttura per il metodo private void `GestisciMenuUtente()`

```

{
    // TODO: implementare il menu utente.
    // Operazioni richieste dalla traccia:
    // - visualizzare catalogo;
    // - aggiungere prodotto al carrello;
    // - visualizzare carrello;
    // - modificare quantità nel carrello;
    // - rimuovere prodotto dal carrello;
    // - svuotare carrello;
    // - confermare acquisto;
    // - visualizzare storico acquisti dell'utente.
    throw new NotImplementedException("Completare il metodo
GestisciMenuUtente.");
}

```

Perfetto, questo metodo è il fulcro dell'esperienza per il cliente del negozio. Visto che ci sono molte opzioni (ben 8 operazioni), il modo migliore per strutturarli senza creare un codice disordinato è usare un ciclo `while` combinato con un costrutto `switch`.

Prima di scrivere il codice, dobbiamo fare una scelta di design importante per l'esame: come identifichiamo l'utente corrente? Visto che la traccia menziona la necessità di "visualizzare storico acquisti dell'utente" e "filtro acquisti per nome

utente", conviene chiedere il nome dell'utente (o simulare un login veloce) non appena si entra nel menu utente, oppure passarlo come parametro.

Assumiamo di chiedere il nome all'inizio del metodo per poter personalizzare il carrello e lo storico.

Ecco una struttura pulita, robusta e con tutte le firme dei metodi inserite al posto giusto:

C#

```
private void GestisciMenuUtente()
{
    Console.Clear();

    Console.WriteLine("=== ACCESSO UTENTE ===");
    Console.Write("Inserisci il tuo nome utente: ");
    string? username = Console.ReadLine();

    if (string.IsNullOrEmpty(username))
    {
        username = "Ospite"; // Fallback di sicurezza se preme invio
    }

    // Qui creiamo o recuperiamo il carrello specifico per l'utente se
necessario,
    // o usiamo quello globale a seconda di come hai strutturato
ServizioNegozio.

    bool tornaAlPrincipale = false;
    while (!tornaAlPrincipale)
    {
        Console.WriteLine($"
--- MENU UTENTE ({username}) ---");
        Console.WriteLine("1. Visualizza Catalogo Prodotti");
        Console.WriteLine("2. Aggiungi Prodotto al Carrello");
        Console.WriteLine("3. Visualizza Carrello");
        Console.WriteLine("4. Modifica Quantità nel Carrello");
        Console.WriteLine("5. Rimuovi Prodotto dal Carrello");
        Console.WriteLine("6. Svuota Carrello");
        Console.WriteLine("7. Conferma Acquisto (Checkout)");
        Console.WriteLine("8. Visualizza Storico Acquisti");
        Console.WriteLine("0. Torna al Menu Principale");
        Console.Write("Seleziona un'operazione: ");
    }
}
```

```
string? scelta = Console.ReadLine()?.Trim();

switch (scelta)
{
    case "1":
        // TODO: Chiamare metodo per visualizzare il catalogo
        // VisualizzaCatalogo();
        break;
    case "2":
        // TODO: Chiamare metodo per aggiungere prodotto tramite codice
        // AggiungiProdottoAlCarrello();
        break;
    case "3":
        // TODO: Chiamare metodo per mostrare gli elementi del carrello
e il totale
        // VisualizzaCarrello();
        break;
    case "4":
        // TODO: Chiamare metodo per modificare la quantità di un
elemento
        // ModificaQuantitaCarrello();
        break;
    case "5":
        // TODO: Chiamare metodo per rimuovere un prodotto dal carrello
        // RimuoviDalCarrello();
        break;
    case "6":
        // TODO: Chiamare metodo per svuotare il carrello
        // SvuotaCarrello();
        break;
    case "7":
        // TODO: Chiamare ServizioNegozio.ConfermaAcquisto(...)
        // ConfermaAcquisto(username);
        break;
    case "8":
        // TODO: Chiamare StoricoAcquisti.FiltraPerUtente(username) e
stamparlo
        // VisualizzaStoricoUtente(username);
        break;
```

```

        case "0":
            tornaAlPrincipale = true;
            Console.WriteLine("Ritorno al menu principale...");
            break;
        default:
            Console.WriteLine("Opzione non valida. Riprova.");
            break;
    }
}
}

```

Perché questa struttura funziona bene per l'esame:

- **Separazione delle responsabilità:** Il menu si occupa solo di instradare l'utente. La logica vera e propria sarà dentro i metodi dedicati (es. `AggiungiProdottoAlCarrello()`), mantenendo il codice pulito e facile da correggere per il professore.
- **Flessibilità:** Usando lo `switch` con le stringhe ("`1`", "`2`"), evitiamo che il programma vada in crash se l'utente digita per errore una lettera al posto di un numero (cosa che succederebbe convertendo brutalmente l'input in un `int`).

Ti torna come impostazione? Se ti piace, vogliamo passare a definire in modo speculare il `GestisciMenuAmministratore()` oppure preferisci iniziare a scrivere la logica di una di queste funzioni dell'utente?

la classe utente ha già un controllo `IsNullOrEmpty`. questo può essere riutilizzato nel metodo `GestisciMenuUtente()`?

Questo è il codice:

```

public class Utente
{
    public string Nome { get; private set; }

    public Utente(string nome)
    {
        if (string.IsNullOrEmpty(nome))
        {
            throw new ArgumentException("Il nome utente non può essere vuoto.");
        }

        Nome = nome.Trim();
    }
}

```

```
}
```

Sì, assolutamente! Anzi, riutilizzarlo è un'ottima pratica di programmazione ad oggetti (evita di duplicare la logica di validazione).

Visto che il costruttore di `Utente` lancia un'eccezione (`ArgumentException`) se il nome non è valido, puoi sfruttare un blocco `try-catch` all'interno di `GestisciMenuUtente()`. Se l'istanza viene creata con successo, significa che l'input era valido e puoi anche usare la proprietà `Nome` (che è già stata pulita con il `.Trim()` nel costruttore!).

Ecco come si integra elegantemente nel tuo metodo:

C#

```
private void GestisciMenuUtente()
{
    Console.Clear();
    Console.WriteLine("=== ACCESSO UTENTE ===");

    Utente? utenteCorrente = null;

    // Ciclo finché non viene inserito un utente valido secondo le regole della
    classe Utente
    while (utenteCorrente == null)
    {
        Console.Write("Inserisci il tuo nome utente: ");
        string? inputNome = Console.ReadLine();

        try
        {
            // Proviamo a istanziare la classe.
            // Se inputNome è vuoto, il costruttore lancerà l'eccezione ed
            entreremo nel catch.
            utenteCorrente = new Utente(inputNome!);
        }
        catch (ArgumentException ex)
        {
            // Intercettiamo l'errore generato dalla classe Utente e lo
            mostriamo
            Console.WriteLine($"Errore: {ex.Message} Riprova.");
        }
    }
}
```

```

    // Da qui in poi siamo SICURI che utenteCorrente non è null e ha un nome
valido
    bool tornaAlPrincipale = false;
    while (!tornaAlPrincipale)
    {
        // Usiamo la proprietà Nome dell'oggetto utenteCorrente, che è già
formattata
        Console.WriteLine($"\\n--- MENU UTENTE ({utenteCorrente.Nome}) ---");
        Console.WriteLine("1. Visualizza Catalogo Prodotti");
        Console.WriteLine("2. Aggiungi Prodotto al Carrello");
        Console.WriteLine("3. Visualizza Carrello");
        Console.WriteLine("4. Modifica Quantità nel Carrello");
        Console.WriteLine("5. Rimuovi Prodotto dal Carrello");
        Console.WriteLine("6. Svuota Carrello");
        Console.WriteLine("7. Conferma Acquisto (Checkout)");
        Console.WriteLine("8. Visualizza Storico Acquisti");
        Console.WriteLine("0. Torna al Menu Principale");
        Console.Write("Seleziona un'operazione: ");

        string? scelta = Console.ReadLine()?.Trim();

        switch (scelta)
        {
            case "1":
                // Spazio per i TODO successivi...
                break;
            // ... altri case ...
            case "0":
                tornaAlPrincipale = true;
                Console.WriteLine("Ritorno al menu principale...");
                break;
            default:
                Console.WriteLine("Opzione non valida. Riprova.");
                break;
        }
    }
}

```

Perché questo approccio piace ai professori:

1. **Incapsulamento rispettato:** La responsabilità di decidere se un nome utente è valido resta dentro la classe `Utente`. Il menu si limita a reagire a quella regola.
2. **Gestione delle eccezioni:** Dimostra una buona padronanza del `try-catch` applicato alla logica di business.

Ti piace questa soluzione? Vuoi che passiamo alla struttura del menu amministratore, o preferisci definire l'equivalente per la classe `Amministratore` (se esiste una classe dedicata) prima?

ho corretto il mio codice. va bene?

```
public class Program
{
    public static void Main()
    {
        // Punto di ingresso della Console App.
        ApplicazioneNegozio applicazione = new ApplicazioneNegozio();
        applicazione.Avvia();
        // TestNegozioOnline.EseguiTuttiITest();
    }
}

public class ApplicazioneNegozio
{
    private readonly CatalogoProdotti catalogoProdotti;
    private readonly CarrelloUtente carrelloUtente;
    private readonly StoricoAcquisti storicoAcquisti;
    private readonly ServizioNegozio servizioNegozio;

    public ApplicazioneNegozio()
    {
        catalogoProdotti = new CatalogoProdotti();
        carrelloUtente = new CarrelloUtente();
        storicoAcquisti = new StoricoAcquisti();
        servizioNegozio = new ServizioNegozio(catalogoProdotti, carrelloUtente,
        storicoAcquisti);

        CaricaDatiIniziali();
    }

    public void Avvia()
```

```

{
    Console.WriteLine("=== BENVENUTO NEL NEGOZIO ONLINE ===");

    bool esci = false;
    while (!esci)
    {
        Console.WriteLine("\n--- MENU PRINCIPALE ---");

        string scelta = ScegliRuolo();
        if (scelta == "utente")
        {
            // Console.WriteLine("\n--- Hai selezionato il ruolo di utente ---");
            GestisciMenuUtente();
        }
        else if (scelta == "amministratore")
        {
            Console.WriteLine("\n--- Hai selezionato il ruolo di amministratore ---");
            // GestisciMenuAmministratore();
        }
        else if (scelta == "esci")
        {
            esci = true;
            Console.WriteLine("\nGrazie per aver utilizzato il negozio online.
Arrivederci!");
        }
        else
        {
            Console.WriteLine("Scelta non valida. Riprova.");
        }
    }
}

private void CaricaDatiIniziali()
{
    // Metodo già implementato: fornisce prodotti di partenza per testare subito il
    sistema.
    catalogoProdotti.AggiungiProdotto(new Prodotto("P001", "Tastiera
meccanica", 79.90m, 10));
}

```

```
        catalogoProdotti.AggiungiProdotto(new Prodotto("P002", "Mouse wireless",  
24.50m, 25));
```

```
        catalogoProdotti.AggiungiProdotto(new Prodotto("P003", "Monitor 24 pollici",  
149.99m, 7));
```

```
        catalogoProdotti.AggiungiProdotto(new Prodotto("P004", "Cavo USB-C",  
9.99m, 40));
```

```
    }
```

```
private string ScegliRuolo()
```

```
{
```

```
    while (true)
```

```
    {
```

```
        Console.WriteLine("\nScegli ruolo (utente/amministratore/esci): ");
```

```
        string? input = Console.ReadLine();
```

```
        if (!string.IsNullOrEmpty(input))
```

```
        {
```

```
            string sceltaFormattata = input.Trim().ToLower();
```

```
            if (sceltaFormattata == "utente" || sceltaFormattata == "amministratore" ||  
sceltaFormattata == "esci")
```

```
            {
```

```
                return sceltaFormattata;
```

```
            }
```

```
        }
```

```
        Console.WriteLine("Opzione non valida. Inserisci esattamente 'utente',  
'amministratore' o 'esci'.");
```

```
    }
```

```
}
```

```
private void GestisciMenuUtente()
```

```
{
```

```
    // TODO: implementare il menu utente.
```

```
    // Operazioni richieste dalla traccia:
```

```
    // - visualizzare catalogo;
```

```
    // - aggiungere prodotto al carrello;
```

```
    // - visualizzare carrello;
```

```
// - modificare quantità nel carrello;  
// - rimuovere prodotto dal carrello;  
// - svuotare carrello;  
// - confermare acquisto;  
// - visualizzare storico acquisti dell'utente.
```

```
Console.WriteLine("=== MENU UTENTE ===");  
Console.Write("Inserisci il tuo nome utente: ");  
string? username = Console.ReadLine();
```

```
Utente utente;  
try  
{  
    utente = new Utente(username);  
}  
catch (ArgumentException)  
{  
    Console.WriteLine("Nome utente non valido. Riprova.");  
    return;  
}
```

```
Console.WriteLine($"Benvenuto, {utente.Nome}!");
```

```
bool tornaAlMenu = false;  
while (!tornaAlMenu){  
    Console.WriteLine("\n--- Quale operazione vuoi eseguire? ---");  
    Console.WriteLine("1. Visualizza catalogo");  
    Console.WriteLine("2. Aggiungi prodotto al carrello");  
    Console.WriteLine("3. Visualizza carrello");  
    Console.WriteLine("4. Modifica quantità nel carrello");  
    Console.WriteLine("5. Rimuovi prodotto dal carrello");  
    Console.WriteLine("6. Svuota carrello");  
    Console.WriteLine("7. Conferma acquisto");  
    Console.WriteLine("8. Visualizza storico acquisti");  
    Console.WriteLine("0. Torna al menu principale");  
    Console.Write("Scelta: ");
```

```
string? scelta = Console.ReadLine();  
switch (scelta)
```

```
{
    case "1":
        MostraCatalogo();
        break;
    case "2":
        // AggiungiProdotto();
        Console.WriteLine("Funzionalità di aggiunta prodotto al carrello non
ancora implementata.");
        break;
    case "3":
        // MostraCarrello();
        Console.WriteLine("Funzionalità di visualizzazione carrello non ancora
implementata.");
        break;
    case "4":
        // ModificaQuantitaCarrello();
        Console.WriteLine("Funzionalità di modifica quantità nel carrello non
ancora implementata.");
        break;
    case "5":
        // RimuoviProdottoCarrello();
        Console.WriteLine("Funzionalità di rimozione prodotto dal carrello non
ancora implementata.");
        break;
    case "6":
        // SvuotaCarrello();
        Console.WriteLine("Funzionalità di svuotamento carrello non ancora
implementata.");
        break;
    case "7":
        // ConfermaAcquisto();
        Console.WriteLine("Funzionalità di conferma acquisto non ancora
implementata.");
        break;
    case "8":
        // VisualizzaStoricoAcquisti();
        Console.WriteLine("Funzionalità di visualizzazione storico acquisti non
ancora implementata.");
        break;
```

```

        case "0":
            tornaAlMenu = true;
            break;
        default:
            Console.WriteLine("Scelta non valida. Riprova.");
            break;
    }
}
// throw new NotImplementedException("Completare il metodo
GestisciMenuUtente.");
}

```

Il tuo codice è strutturato molto bene. Hai integrato perfettamente la validazione della classe `Utente` tramite il blocco `try-catch` all'interno di `GestisciMenuUtente()`. L'inizializzazione dei campi tramite costruttore con `CaricaDatiIniziali()` garantisce che l'applicazione sia pronta per i test fin dal primo avvio. Ci sono solo due appunti importanti da considerare per far sì che tutto funzioni come previsto nei prossimi passi:

1. Il comportamento del `return` in caso di errore

Nel blocco `catch` di `GestisciMenuUtente()`, hai inserito un `return;`:
C#

```

catch (ArgumentException)
{
    Console.WriteLine("Nome utente non valido. Riprova.");
    return; // <--- Questo fa uscire dal menu utente e torna al menu principale
}

```

Se l'utente sbaglia a digitare il nome (ad esempio preme invio per errore), viene rispedito direttamente al **Menu Principale**. Se l'intento è costringerlo a inserire un nome valido senza farlo uscire dall'area utente, potresti racchiudere solo la richiesta del nome in un piccolo ciclo `while`, in questo modo:

C#

```

Utente? utente = null;
while (utente == null)
{
    Console.Write("Inserisci il tuo nome utente: ");
    string? username = Console.ReadLine();
    try
    {
        utente = new Utente(username);
    }
}

```

```

    }

    catch (ArgumentException)
    {
        Console.WriteLine("Nome utente non valido. Riprova.");
    }
}

```

2. Sincronizzazione del costruttore di ServizioNegozio

Nel costruttore passi tre parametri: `servizioNegozio = new`

`ServizioNegozio(catalogoProdotti, carrelloUtente, storicoAcquisti);`

Nello schema iniziale avevamo ipotizzato un costruttore a due parametri (`CatalogoProdotti`, `StoricoAcquisti`), ipotizzando che il carrello venisse passato direttamente al metodo `ConfermaAcquisto(utente, carrello)`. Avendo incluso il carrello direttamente nel servizio (scelta validissima), assicurati solo che la firma del costruttore dentro la classe `ServizioNegozio` accetti esattamente questi tre parametri.

Prossimo Passo: Implementare MostraCatalogo()

Visto che nel case "1" hai già inserito la chiamata a `MostraCatalogo()`, passiamo a scrivere questo metodo all'interno di `ApplicazioneNegozio`.

Dovrà mostrare a schermo i prodotti presenti nel catalogo in modo leggibile (incolonnati). Ecco come puoi implementarlo:

C#

```

private void MostraCatalogo()
{
    Console.WriteLine("\n=== CATALOGO PRODOTTI ===");

    // Recuperiamo la lista dal catalogo
    var prodotti = catalogoProdotti.OttieniCatalogo();

    if (prodotti.Count == 0)
    {
        Console.WriteLine("Il catalogo è vuoto.");
        return;
    }

    // Intestazione tabella formattata
    Console.WriteLine(string.Format("{0,-10} {1,-25} {2,-12} {3,-12}", "Codice",
"Nome", "Prezzo", "Disponibile"));

```

```

        Console.WriteLine(new string('-', 64));

        foreach (var p in prodotti)
        {
            // Stampa formattata: allineamento a sinistra e valuta per il prezzo
            Console.WriteLine(string.Format("{0,-10} {1,-25} {2,-12:C} {3,-12}",
                p.Codice,
                p.Nome,
                p.Prezzo,
                p.QuantitaDisponibile));
        }
    }
}

```

Vuoi procedere inserendo questo metodo o preferisci prima definire lo scheletro di `GestisciMenuAmministratore()` per completare la prima fase dei menu?

il metodo `mostraCatalogo` è già implementato, passiamo al metodo `gestisciMenuAmministratore()`.

```

private void GestisciMenuAmministratore()
{
    // TODO: implementare il menu amministratore.
    // Operazioni richieste dalla traccia:
    // - visualizzare catalogo completo;
    // - aggiungere prodotto;
    // - eliminare prodotto;
    // - modificare prezzo;
    // - aumentare o diminuire quantità disponibile;
    // - visualizzare tutti gli acquisti;
    // - visualizzare quantità iniziale, vendita e disponibile per prodotto.
    throw new NotImplementedException("Completare il metodo
GestisciMenuAmministratore.");
}

```

Perfetto, passiamo all'interfaccia per l'amministratore. Qui le operazioni si concentrano sulla gestione avanzata del magazzino e sul monitoraggio delle vendite.

Seguendo la stessa struttura robusta che abbiamo usato per il menu utente (ciclo `while` e costruito `switch` su stringhe), possiamo mappare tutte e 7 le richieste della traccia.

Ecco lo scheletro ideale per `GestisciMenuAmministratore()` all'interno della tua classe `ApplicazioneNegozio`:
C#


```

private void GestisciMenuAmministratore()
{
    Console.Clear();
    Console.WriteLine("=== AREA AMMINISTRATORE ===");

    bool tornaAlMenuPrincipale = false;
    while (!tornaAlMenuPrincipale)
    {
        Console.WriteLine("\n--- PANNELLO DI CONTROLLO ADMIN ---");
        Console.WriteLine("1. Visualizza catalogo completo");
        Console.WriteLine("2. Aggiungi nuovo prodotto al catalogo");
        Console.WriteLine("3. Elimina prodotto dal catalogo");
        Console.WriteLine("4. Modifica prezzo di un prodotto");
        Console.WriteLine("5. Modifica quantità disponibile (Incolla/Preleva magazzino)");
        Console.WriteLine("6. Visualizza tutti gli acquisti effettuati (Storico globale)");
        Console.WriteLine("7. Visualizza report vendite (Qta Iniziale/Venduta/Disponibile)");
        Console.WriteLine("0. Torna al menu principale");
        Console.Write("Scelta: ");

        string? scelta = Console.ReadLine()?.Trim();

        switch (scelta)
        {
            case "1":
                // Utilizziamo il metodo che hai già implementato
                MostraCatalogo();
                break;
            case "2":
                // TODO: Chiamare metodo per inserimento nuovo Prodotto
                Console.WriteLine("Funzionalità di aggiunta prodotto non ancora implementata.");
                break;
            case "3":
                // TODO: Chiamare metodo per eliminazione tramite Codice
                Console.WriteLine("Funzionalità di eliminazione prodotto non ancora implementata.");

```

```

        break;
    case "4":
        // TODO: Chiamare metodo per cambio prezzo
        Console.WriteLine("Funzionalità di modifica prezzo non ancora implementata.");
        break;
    case "5":
        // TODO: Chiamare metodo per alterare la quantità disponibile
        Console.WriteLine("Funzionalità di modifica quantità non ancora implementata.");
        break;
    case "6":
        // TODO: Chiamare metodo per mostrare TUTTI gli acquisti nello storico
        Console.WriteLine("Funzionalità di visualizzazione storico globale non ancora implementata.");
        break;
    case "7":
        // TODO: Generare il report richiesto usando il modello ReportProdotto
        Console.WriteLine("Funzionalità di report vendite non ancora implementata.");
        break;
    case "0":
        tornaAlMenuPrincipale = true;
        break;
    default:
        Console.WriteLine("Scelta non valida. Riprova.");
        break;
    }
}
}

```

Un dettaglio architetturale per i prossimi passi

Nota le operazioni **4** e **5** (modificare prezzo e aumentare/diminuire quantità).

All'inizio avevamo definito nel file di partenza questo metodo nel catalogo: `bool`

```

ModificaProdotto(string codice, string nuovoNome, decimal nuovoPrezzo, int
nuovaQuantita)

```

Quando andremo a implementare la logica dietro i case "4" e "5", potremo decidere se usare quel metodo unico passandogli i vecchi valori per i campi che non vogliamo cambiare, oppure se aggiungere dei piccoli metodi specifici nel catalogo per non portarci dietro troppi parametri. Ci ragioneremo insieme quando toccheremo la logica del catalogo.

Ora che la struttura di base di entrambi i menu (Fase 1) è completata con successo, da dove vogliamo iniziare ad attaccare i **TODO** della logica?

1. Gestiamo l'**aggiunta di un prodotto al carrello da parte dell'utente** (Case "2" del menu utente)?
2. Oppure preferisci completare prima le funzioni di **inserimento/modifica dei prodotti lato amministratore**?

ecco il mio codice e dimmi cosa manca e spiegami passo passo come implementare. è importantissimo che non mi cambi firma, nomi, parametri e tipo di ritorno dei metodi.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
/*
```

```
* TEMPLATE ESAME C# - NEGOZIO ONLINE
```

```
*
```

```
* Regola scelta per il template:
```

```
* - i metodi di visualizzazione sono già implementati, così lo studente può concentrarsi
```

```
* sulle operazioni richieste dalla traccia.
```

```
* - i metodi operazionali contengono TODO guidati: lo studente deve completarli senza
```

```
* modificare firma, nome, parametri o tipo di ritorno.
```

```
*
```

```
* Vincolo richiesto: tutto il codice è in un unico file .cs e senza namespace.
```

```
*/
```

```
public class Program
```

```
{
```

```
    public static void Main()
```

```
    {
```

```
        // Punto di ingresso della Console App.
```

```
        ApplicazioneNegozio applicazione = new ApplicazioneNegozio();
```

```
        applicazione.Avvia();
        // TestNegozioOnline.EseguiTuttiITest();
    }
}
```

```
public class ApplicazioneNegozio
{
    private readonly CatalogoProdotti catalogoProdotti;
    private readonly CarrelloUtente carrelloUtente;
    private readonly StoricoAcquisti storicoAcquisti;
    private readonly ServizioNegozio servizioNegozio;

    public ApplicazioneNegozio()
    {
        catalogoProdotti = new CatalogoProdotti();
        carrelloUtente = new CarrelloUtente();
        storicoAcquisti = new StoricoAcquisti();
        servizioNegozio = new ServizioNegozio(catalogoProdotti, carrelloUtente,
        storicoAcquisti);

        CaricaDatiIniziali();
    }

    public void Avvia()
    {
        Console.WriteLine("=== BENVENUTO NEL NEGOZIO ONLINE ===");

        bool esci = false;
        while (!esci)
        {
            Console.WriteLine("\n--- MENU PRINCIPALE ---");

            string scelta = ScegliRuolo();
            if (scelta == "utente")
            {
                // Console.WriteLine("\n--- Hai selezionato il ruolo di utente ---");
                GestisciMenuUtente();
            }
            else if (scelta == "amministratore")
```

```

        {
            // Console.WriteLine("\n--- Hai selezionato il ruolo di amministratore ---");
            GestisciMenuAmministratore();
        }
        else if (scelta == "esci")
        {
            esci = true;
            Console.WriteLine("\nGrazie per aver utilizzato il negozio online.
Arrivederci!");
        }
        else
        {
            Console.WriteLine("Scelta non valida. Riprova.");
        }
    }
}

```

```

private void CaricaDatiIniziali()
{
    // Metodo già implementato: fornisce prodotti di partenza per testare subito il
    sistema.
    catalogoProdotti.AggiungiProdotto(new Prodotto("P001", "Tastiera
meccanica", 79.90m, 10));
    catalogoProdotti.AggiungiProdotto(new Prodotto("P002", "Mouse wireless",
24.50m, 25));
    catalogoProdotti.AggiungiProdotto(new Prodotto("P003", "Monitor 24 pollici",
149.99m, 7));
    catalogoProdotti.AggiungiProdotto(new Prodotto("P004", "Cavo USB-C",
9.99m, 40));
}

```

```

private string ScegliRuolo()
{
    while (true)
    {
        Console.Write("\nScegli ruolo (utente/amministratore/esci): ");
        string? input = Console.ReadLine();
    }
}

```

```

        if (!string.IsNullOrEmpty(input))
        {
            string sceltaFormattata = input.Trim().ToLower();

            if (sceltaFormattata == "utente" || sceltaFormattata == "amministratore" ||
sceltaFormattata == "esci")
            {
                return sceltaFormattata;
            }
        }

        Console.WriteLine("Opzione non valida. Inserisci esattamente 'utente',
'amministratore' o 'esci'.");
    }
}

```

```

private void GestisciMenuUtente()
{
    // TODO: implementare il menu utente.
    // Operazioni richieste dalla traccia:
    // - visualizzare catalogo;
    // - aggiungere prodotto al carrello;
    // - visualizzare carrello;
    // - modificare quantità nel carrello;
    // - rimuovere prodotto dal carrello;
    // - svuotare carrello;
    // - confermare acquisto;
    // - visualizzare storico acquisti dell'utente.
}

```

```

Console.WriteLine("=== MENU UTENTE ===");

```

```

Utente? utente = null;
while (utente == null)
{
    Console.Write("Inserisci il tuo nome utente: ");
    string? username = Console.ReadLine();
    try
    {
        utente = new Utente(username);
    }
}

```

```

    }
    catch (ArgumentException)
    {
        Console.WriteLine("Nome utente non valido. Riprova.");
    }
}

```

```

Console.WriteLine($"Benvenuto, {utente.Nome}!");

```

```

bool tornaAlMenu = false;

```

```

while (!tornaAlMenu){

```

```

    Console.WriteLine("\n--- Quale operazione vuoi eseguire? ---");

```

```

    Console.WriteLine("1. Visualizza catalogo");

```

```

    Console.WriteLine("2. Aggiungi prodotto al carrello");

```

```

    Console.WriteLine("3. Visualizza carrello");

```

```

    Console.WriteLine("4. Modifica quantità nel carrello");

```

```

    Console.WriteLine("5. Rimuovi prodotto dal carrello");

```

```

    Console.WriteLine("6. Svuota carrello");

```

```

    Console.WriteLine("7. Conferma acquisto");

```

```

    Console.WriteLine("8. Visualizza storico acquisti");

```

```

    Console.WriteLine("0. Torna al menu principale");

```

```

    Console.Write("Scelta: ");

```

```

string? scelta = Console.ReadLine();

```

```

switch (scelta)

```

```

{

```

```

    case "1":

```

```

        MostraCatalogo();

```

```

        break;

```

```

    case "2":

```

```

        Console.WriteLine("Funzionalità di aggiunta prodotto al carrello non  
ancora implementata.");

```

```

        break;

```

```

    case "3":

```

```

        MostraCarrello();

```

```

        break;

```

```

    case "4":

```

```

        Console.WriteLine("Funzionalità di modifica quantità nel carrello non  
ancora implementata.");

```

```

        break;
    case "5":
        Console.WriteLine("Funzionalità di rimozione prodotto dal carrello non ancora implementata.");
        break;
    case "6":
        carrelloUtente.SvuotaCarrello();
        Console.WriteLine("Carrello svuotato.");
        break;
    case "7":
        Console.WriteLine("Funzionalità di conferma acquisto non ancora implementata.");
        break;
    case "8":
        MostraStoricoUtente();
        break;
    case "0":
        tornaAlMenu = true;
        break;
    default:
        Console.WriteLine("Scelta non valida. Riprova.");
        break;
    }
}
// throw new NotImplementedException("Completare il metodo GestisciMenuUtente.");
}

```

```

private void GestisciMenuAmministratore()
{

```

```

    Console.Clear();
    Console.WriteLine("=== AREA AMMINISTRATORE ===");

```

```

    bool tornaAlMenuPrincipale = false;
    while (!tornaAlMenuPrincipale)
    {

```

```

        Console.WriteLine("\n--- PANNELLO DI CONTROLLO ADMIN ---");
        Console.WriteLine("1. Visualizza catalogo completo");
        Console.WriteLine("2. Aggiungi nuovo prodotto al catalogo");
    }
}

```



```

        Console.WriteLine("3. Elimina prodotto dal catalogo");
        Console.WriteLine("4. Modifica prezzo di un prodotto");
        Console.WriteLine("5. Modifica quantità disponibile (Incolla/Preleva magazzino)");
        Console.WriteLine("6. Visualizza tutti gli acquisti effettuati (Storico globale)");
        Console.WriteLine("7. Visualizza report vendite (Qta Iniziale/Venduta/Disponibile)");
        Console.WriteLine("0. Torna al menu principale");
        Console.Write("Scelta: ");

        string? scelta = Console.ReadLine()?.Trim();

        switch (scelta)
        {
            case "1":
                // Utilizziamo il metodo che hai già implementato
                MostraCatalogo();
                break;
            case "2":
                // Aggiungi nuovo prodotto al catalogo (metodo CatalogoProdotti.AggiungiProdotto è implementato)
                Console.Write("Inserisci codice prodotto: ");
                string? codice = Console.ReadLine();
                Console.Write("Inserisci nome prodotto: ");
                string? nome = Console.ReadLine();
                decimal prezzo = LeggiPrezzoPositivo("Inserisci prezzo (es. 19.99): ");
                int quantita = LeggiInteroPositivo("Inserisci quantità disponibile: ");

                if (string.IsNullOrEmpty(codice) || string.IsNullOrEmpty(nome))
                {
                    Console.WriteLine("Codice o nome prodotto non valido.");
                    break;
                }

                try
                {

```

```

        catalogoProdotti.AggiungiProdotto(new Prodotto(codice.Trim(),
nome.Trim(), prezzo, quantita));
        Console.WriteLine("Prodotto aggiunto al catalogo.");
    }
    catch (Exception ex)
    {
        Console.WriteLine("Impossibile aggiungere il prodotto: " +
ex.Message);
    }
    break;
case "3":
    // TODO: Chiamare metodo per eliminazione tramite Codice
    Console.WriteLine("Funzionalità di eliminazione prodotto non ancora
implementata.");
    break;
case "4":
    // TODO: Chiamare metodo per cambio prezzo
    Console.WriteLine("Funzionalità di modifica prezzo non ancora
implementata.");
    break;
case "5":
    // TODO: Chiamare metodo per alterare la quantità disponibile
    Console.WriteLine("Funzionalità di modifica quantità non ancora
implementata.");
    break;
case "6":
    List<Acquisto> tuttiGliAcquisti =
storicoAcquisti.OttieniTuttiGliAcquisti();
    if (tuttiGliAcquisti.Count == 0)
    {
        Console.WriteLine("Nessun acquisto registrato.");
    }
    else
    {
        Console.WriteLine("=== TUTTI GLI ACQUISTI ===");
        foreach (Acquisto a in tuttiGliAcquisti)
        {
            servizioNegozio.StampaAcquisto(a);
        }
    }
}

```

```

        }
        break;
    case "7":
        servizioNegozio.StampaReportProdotti();
        break;
    case "0":
        tornaAlMenuPrincipale = true;
        break;
    default:
        Console.WriteLine("Scelta non valida. Riprova.");
        break;
    }
}
}

```

```

private void MostraCatalogo()
{
    // Metodo già implementato: mostra a video tutti i prodotti del catalogo.
    List<Prodotto> prodotti = catalogoProdotti.OttieniTuttiIProdotti();

    Console.WriteLine();
    Console.WriteLine("=== CATALOGO PRODOTTI ===");

    if (prodotti.Count == 0)
    {
        Console.WriteLine("Il catalogo è vuoto.");
        return;
    }

    foreach (Prodotto prodotto in prodotti)
    {
        Console.WriteLine(
            prodotto.CodiceProdotto + " - " +
            prodotto.Nome + " - " +
            prodotto.Prezzo.ToString("0.00") + " euro - " +
            "Disponibili: " + prodotto.QuantitaDisponibile);
    }
}

```

```

private void MostraCarrello()
{
    // Metodo già implementato: mostra contenuto del carrello e totale corrente.
    List<ElementoCarrello> elementi = carrelloUtente.OttieniElementi();

    Console.WriteLine();
    Console.WriteLine("=== CARRELLO ===");

    if (elementi.Count == 0)
    {
        Console.WriteLine("Il carrello è vuoto.");
        return;
    }

    foreach (ElementoCarrello elemento in elementi)
    {
        Console.WriteLine(
            elemento.ProdottoSelezionato.CodiceProdotto + " - " +
            elemento.ProdottoSelezionato.Nome + " - " +
            "Quantità: " + elemento.QuantitaScelta + " - " +
            "Prezzo unitario: " + elemento.PrezzoUnitario.ToString("0.00") + " euro - "
+
            "Parziale: " + elemento.CalcolaTotaleParziale().ToString("0.00") + "
euro");
    }

    Console.WriteLine("Totale carrello: " +
carrelloUtente.CalcolaTotale().ToString("0.00") + " euro");
}

private void MostraStoricoUtente()
{
    // Metodo già implementato: chiede un nome e mostra gli acquisti collegati.
    Console.Write("Inserisci nome utente: ");
    string? nomeUtente = Console.ReadLine();

    if (string.IsNullOrEmpty(nomeUtente))
    {
        Console.WriteLine("Nome utente non valido.");
    }
}

```

```
    return;  
}
```

```
    List<Acquisto> acquistiUtente =  
storicoAcquisti.OttieniAcquistiPerUtente(nomeUtente);
```

```
    Console.WriteLine();  
    Console.WriteLine("=== STORICO ACQUISTI DI " + nomeUtente.Trim() + "  
===");
```

```
    if (acquistiUtente.Count == 0)  
    {  
        Console.WriteLine("Nessun acquisto trovato per questo utente.");  
        return;  
    }
```

```
    foreach (Acquisto acquisto in acquistiUtente)  
    {  
        servizioNegozio.StampaAcquisto(acquisto);  
    }  
}
```

```
private int LeggiInteroPositivo(string messaggio)  
{  
    // Legge da console un numero intero positivo > 0.  
    while (true)  
    {  
        Console.Write(messaggio);  
        string? input = Console.ReadLine();  
  
        if (int.TryParse(input, out int valore) && valore > 0)  
        {  
            return valore;  
        }  
  
        Console.WriteLine("Inserisci un numero intero positivo valido.");  
    }  
}
```

```

private decimal LeggiPrezzoPositivo(string messaggio)
{
    while (true)
    {
        Console.Write(messaggio);
        string? input = Console.ReadLine();

        if (decimal.TryParse(input, out decimal valore) && valore > 0m)
        {
            return valore;
        }

        Console.WriteLine("Inserisci un prezzo positivo valido (maggiore di 0).");
    }
}

```

```

public interface IGestioneCatalogo
{
    void AggiungiProdotto(Prodotto prodotto);
    bool EliminaProdotto(string codiceProdotto);
    Prodotto? CercaProdottoPerCodice(string codiceProdotto);
    List<Prodotto> OttieniTuttiIProdotti();
    bool ModificaPrezzoProdotto(string codiceProdotto, decimal nuovoPrezzo);
    bool ModificaQuantitaProdotto(string codiceProdotto, int variazioneQuantita);
}

```

```

public interface IGestioneCarrello
{
    bool AggiungiAlCarrello(Prodotto prodotto, int quantita);
    bool ModificaQuantitaNelCarrello(string codiceProdotto, int nuovaQuantita);
    bool RimuoviDalCarrello(string codiceProdotto);
    void SvuotaCarrello();
    decimal CalcolaTotale();
    List<ElementoCarrello> OttieniElementi();
}

```

```

public interface IGestioneAcquisti
{

```

```
void RegistraAcquisto(Acquisto acquisto);  
List<Acquisto> OttieniTuttiGliAcquisti();  
List<Acquisto> OttieniAcquistiPerUtente(string nomeUtente);  
}
```

```
public class Utente  
{  
    public string Nome { get; private set; }  
  
    public Utente(string nome)  
    {  
        if (string.IsNullOrEmpty(nome))  
        {  
            throw new ArgumentException("Il nome utente non può essere vuoto.");  
        }  
  
        Nome = nome.Trim();  
    }  
}
```

```
public class Prodotto  
{  
    public string CodiceProdotto { get; private set; }  
    public string Nome { get; private set; }  
    public decimal Prezzo { get; private set; }  
    public int QuantitaDisponibile { get; private set; }  
    public int QuantitaIniziale { get; private set; }  
  
    public Prodotto(string codiceProdotto, string nome, decimal prezzo, int  
quantitaDisponibile)  
    {  
        CodiceProdotto = codiceProdotto;  
        Nome = nome;  
        Prezzo = prezzo;  
        QuantitaDisponibile = quantitaDisponibile;  
        QuantitaIniziale = quantitaDisponibile;  
    }  
  
    public void CambiaPrezzo(decimal nuovoPrezzo)
```

```

{
    // Metodo già implementato: centralizza la validazione del prezzo.
    if (nuovoPrezzo <= 0)
    {
        throw new ArgumentException("Il prezzo deve essere maggiore di zero.");
    }

    Prezzo = nuovoPrezzo;
}

public void CambiaQuantita(int variazioneQuantita)
{
    // Metodo già implementato: impedisce di portare il magazzino sotto zero.
    int nuovaQuantita = QuantitaDisponibile + variazioneQuantita;

    if (nuovaQuantita < 0)
    {
        throw new InvalidOperationException("La quantità disponibile non può diventare negativa.");
    }

    QuantitaDisponibile = nuovaQuantita;
}

public int CalcolaQuantitaVenduta()
{
    // Metodo già implementato: serve per il report amministratore.
    return QuantitaIniziale - QuantitaDisponibile;
}
}

public class ElementoCarrello
{
    public Prodotto ProdottoSelezionato { get; private set; }
    public int QuantitaScelta { get; private set; }
    public decimal PrezzoUnitario { get; private set; }

    public ElementoCarrello(Prodotto prodottoSelezionato, int quantitaScelta)
    {

```



```
    ProdottoSelezionato = prodottoSelezionato;  
    QuantitaScelta = quantitaScelta;  
    PrezzoUnitario = prodottoSelezionato.Prezzo;  
}
```

```
public decimal CalcolaTotaleParziale()  
{  
    // Metodo già implementato: evita di duplicare il calcolo del parziale.  
    return PrezzoUnitario * QuantitaScelta;  
}
```

```
public void CambiaQuantitaScelta(int nuovaQuantita)  
{  
    // TODO: validare che la nuova quantità sia maggiore di zero.  
    // Se è valida, aggiornare QuantitaScelta.  
    // Se non è valida, lanciare ArgumentException con un messaggio  
    comprensibile.  
    throw new NotImplementedException("Completare il metodo  
    CambiaQuantitaScelta.");  
}  
}
```

```
public class Acquisto  
{  
    public Utente Utente { get; private set; }  
    public string NomeUtente  
    {  
        get { return Utente.Nome; }  
    }  
  
    public List<ElementoAcquistato> ProdottiAcquistati { get; private set; }  
    public decimal TotaleOrdine { get; private set; }  
    public DateTime DataAcquisto { get; private set; }  
  
    public Acquisto(Utente utente, List<ElementoAcquistato> prodottiAcquistati)  
    {  
        Utente = utente;  
        ProdottiAcquistati = prodottiAcquistati;  
        DataAcquisto = DateTime.Now;  
    }  
}
```

```

        TotaleOrdine = CalcolaTotaleOrdine();
    }

    private decimal CalcolaTotaleOrdine()
    {
        // Metodo già implementato: somma tutti i parziali dei prodotti acquistati.
        return ProdottiAcquistati.Sum(prodotto => prodotto.TotaleParziale);
    }
}

public class ElementoAcquistato
{
    public string CodiceProdotto { get; private set; }
    public string NomeProdotto { get; private set; }
    public int QuantitaAcquistata { get; private set; }
    public decimal PrezzoUnitario { get; private set; }
    public decimal TotaleParziale { get; private set; }

    public ElementoAcquistato(string codiceProdotto, string nomeProdotto, int
quantitaAcquistata, decimal prezzoUnitario)
    {
        CodiceProdotto = codiceProdotto;
        NomeProdotto = nomeProdotto;
        QuantitaAcquistata = quantitaAcquistata;
        PrezzoUnitario = prezzoUnitario;
        TotaleParziale = prezzoUnitario * quantitaAcquistata;
    }
}

public class CatalogoProdotti : IGestioneCatalogo
{
    private readonly List<Prodotto> prodotti;

    public CatalogoProdotti()
    {
        prodotti = new List<Prodotto>();
    }

    public void AggiungiProdotto(Prodotto prodotto)

```

```

{
    // Metodo già implementato: evita codici duplicati nel catalogo.
    bool codiceGiaPresente = prodotti.Any(p => p.CodiceProdotto ==
prodotto.CodiceProdotto);

    if (codiceGiaPresente)
    {
        throw new InvalidOperationException("Esiste già un prodotto con lo stesso
codice.");
    }

    prodotti.Add(prodotto);
}

public bool EliminaProdotto(string codiceProdotto)
{
    // TODO: cercare il prodotto tramite codice e rimuoverlo dalla lista.
    // Restituire true se il prodotto è stato eliminato, false se non esiste.
    throw new NotImplementedException("Completare il metodo
EliminaProdotto.");
}

public Prodotto? CercaProdottoPerCodice(string codiceProdotto)
{
    // Metodo già implementato: ricerca case-insensitive per rendere più comodo
l'input da console.
    return prodotti.FirstOrDefault(prodotto =>
        prodotto.CodiceProdotto.Equals(codiceProdotto,
StringComparison.OrdinalIgnoreCase));
}

public List<Prodotto> OttieniTuttiIProdotti()
{
    // Metodo già implementato: restituisce una copia per proteggere la lista
interna.
    return new List<Prodotto>(prodotti);
}

public bool ModificaPrezzoProdotto(string codiceProdotto, decimal nuovoPrezzo)

```

```

    {
        // TODO: trovare il prodotto e chiamare
        prodotto.CambiaPrezzo(nuovoPrezzo).
        // Restituire false se il codice non esiste.
        throw new NotImplementedException("Completare il metodo
        ModificaPrezzoProdotto.");
    }

    public bool ModificaQuantitaProdotto(string codiceProdotto, int
    variazioneQuantita)
    {
        // TODO: trovare il prodotto e chiamare
        prodotto.CambiaQuantita(variazioneQuantita).
        // La variazione può essere positiva o negativa, ma il magazzino non deve
        scendere sotto zero.
        throw new NotImplementedException("Completare il metodo
        ModificaQuantitaProdotto.");
    }
}

public class CarrelloUtente : IGestioneCarrello
{
    private readonly List<ElementoCarrello> elementiCarrello;

    public CarrelloUtente()
    {
        elementiCarrello = new List<ElementoCarrello>();
    }

    public bool AggiungiAlCarrello(Prodotto prodotto, int quantita)
    {
        // TODO: completare l'aggiunta al carrello.
        // Regole:
        // - rifiutare quantità <= 0;
        // - rifiutare quantità maggiore della disponibilità di magazzino;
        // - se il prodotto è già presente, aumentare la quantità esistente;
        // - controllare che quantità esistente + quantità richiesta non superi il
        magazzino.
    }
}

```

```
        throw new NotImplementedException("Completare il metodo  
AggiungiAlCarrello.");  
    }
```

```
    public bool ModificaQuantitaNelCarrello(string codiceProdotto, int  
nuovaQuantita)  
    {  
        // TODO: trovare l'elemento del carrello e modificarne la quantità.  
        // Regole:  
        // - nuovaQuantita deve essere > 0;  
        // - nuovaQuantita non deve superare la disponibilità del prodotto.  
        throw new NotImplementedException("Completare il metodo  
ModificaQuantitaNelCarrello.");  
    }
```

```
    public bool RimuoviDalCarrello(string codiceProdotto)  
    {  
        // TODO: rimuovere dal carrello l'elemento con il codice indicato.  
        // Restituire true se rimosso, false se non trovato.  
        throw new NotImplementedException("Completare il metodo  
RimuoviDalCarrello.");  
    }
```

```
    public void SvuotaCarrello()  
    {  
        // Metodo già implementato: cancella tutti gli elementi del carrello.  
        elementiCarrello.Clear();  
    }
```

```
    public decimal CalcolaTotale()  
    {  
        // Metodo già implementato: ricalcola sempre il totale dai parziali correnti.  
        return elementiCarrello.Sum(elemento => elemento.CalcolaTotaleParziale());  
    }
```

```
    public List<ElementoCarrello> OttieniElementi()  
    {  
        // Metodo già implementato: restituisce una copia per evitare modifiche  
        esterne dirette.
```

```

        return new List<ElementoCarrello>(elementiCarrello);
    }
}

public class StoricoAcquisti : IGestioneAcquisti
{
    private readonly List<Acquisto> acquisti;

    public StoricoAcquisti()
    {
        acquisti = new List<Acquisto>();
    }

    public void RegistraAcquisto(Acquisto acquisto)
    {
        // Metodo già implementato: conserva l'acquisto in memoria durante
        // l'esecuzione.
        acquisti.Add(acquisto);
    }

    public List<Acquisto> OttieniTuttiGliAcquisti()
    {
        // Metodo già implementato: restituisce una copia dello storico.
        return new List<Acquisto>(acquisti);
    }

    public List<Acquisto> OttieniAcquistiPerUtente(string nomeUtente)
    {
        // TODO: filtrare gli acquisti per nome utente.
        // Consiglio: usare StringComparison.OrdinalIgnoreCase per ignorare
        // maiuscole/minuscole.
        throw new NotImplementedException("Completare il metodo
        OttieniAcquistiPerUtente.");
    }
}

public class ServizioNegozio
{
    private readonly CatalogoProdotti catalogoProdotti;

```

```
private readonly CarrelloUtente carrelloUtente;  
private readonly StoricoAcquisti storicoAcquisti;
```

```
public ServizioNegozio(CatalogoProdotti catalogoProdotti, CarrelloUtente  
carrelloUtente, StoricoAcquisti storicoAcquisti)
```

```
{  
    this.catalogoProdotti = catalogoProdotti;  
    this.carrelloUtente = carrelloUtente;  
    this.storicoAcquisti = storicoAcquisti;  
}
```

```
public bool AggiungiProdottoAlCarrello(string codiceProdotto, int quantita)
```

```
{  
    // TODO: cercare il prodotto nel catalogo e delegare a  
carrelloUtente.AggiungiAlCarrello.  
    // Restituire false se il prodotto non esiste o se la quantità non è valida.  
    throw new NotImplementedException("Completare il metodo  
AggiungiProdottoAlCarrello.");  
}
```

```
public Acquisto ConfermaAcquisto(Utente utente)
```

```
{  
    // TODO: completare la conferma dell'acquisto.  
    // Regole richieste dalla traccia:  
    // - impedire l'acquisto se il carrello è vuoto;  
    // - ricontrollare che ogni quantità sia valida e disponibile in magazzino;  
    // - creare gli ElementoAcquistato partendo dagli elementi del carrello;  
    // - diminuire la quantità disponibile dei prodotti acquistati;  
    // - registrare l'acquisto nello storico;  
    // - svuotare il carrello dopo un acquisto completato;  
    // - creare e restituire un Acquisto associato all'Utente ricevuto.  
    throw new NotImplementedException("Completare il metodo  
ConfermaAcquisto.");  
}
```

```
public List<ReportProdotto> CreaReportProdotti()
```

```
{  
    // Metodo già implementato: prepara il report richiesto per l'amministratore.  
    return catalogoProdotti.OttieniTuttiIProdotti()
```

```

        .Select(prodotto => new ReportProdotto(
            prodotto.CodiceProdotto,
            prodotto.Nome,
            prodotto.QuantitaIniziale,
            prodotto.CalcolaQuantitaVenduta(),
            prodotto.QuantitaDisponibile))
        .ToList();
    }

    public void StampaAcquisto(Acquisto acquisto)
    {
        // Metodo già implementato: mostra i dettagli di un acquisto completato.
        Console.WriteLine("-----");
        Console.WriteLine("Utente: " + acquisto.NomeUtente);
        Console.WriteLine("Data: " + acquisto.DataAcquisto.ToString("dd/MM/yyyy
HH:mm"));
        Console.WriteLine("Prodotti acquistati:");

        foreach (ElementoAcquistato elemento in acquisto.ProdottiAcquistati)
        {
            Console.WriteLine(
                "- " + elemento.CodiceProdotto + " - " +
                elemento.NomeProdotto + " - " +
                "Quantità: " + elemento.QuantitaAcquistata + " - " +
                "Prezzo unitario: " + elemento.PrezzoUnitario.ToString("0.00") + " euro - "
+
                "Parziale: " + elemento.TotaleParziale.ToString("0.00") + " euro");
        }

        Console.WriteLine("Totale ordine: " + acquisto.TotaleOrdine.ToString("0.00") +
" euro");
    }

    public void StampaReportProdotti()
    {
        // Metodo già implementato: mostra il report quantità richiesto
all'amministratore.
        List<ReportProdotto> report = CreaReportProdotti();
    }

```



```
Console.WriteLine();
Console.WriteLine("=== REPORT PRODOTTI ===");
```

```
if (report.Count == 0)
{
    Console.WriteLine("Nessun prodotto presente nel catalogo.");
    return;
}
```

```
foreach (ReportProdotto riga in report)
{
    Console.WriteLine(
        riga.CodiceProdotto + " - " +
        riga.NomeProdotto + " - " +
        "Iniziale: " + riga.QuantitaIniziale + " - " +
        "Venduta: " + riga.QuantitaVenduta + " - " +
        "Disponibile: " + riga.QuantitaDisponibile);
}
}
```

```
public class ReportProdotto
```

```
{
    public string CodiceProdotto { get; private set; }
    public string NomeProdotto { get; private set; }
    public int QuantitaIniziale { get; private set; }
    public int QuantitaVenduta { get; private set; }
    public int QuantitaDisponibile { get; private set; }
```

```
    public ReportProdotto(string codiceProdotto, string nomeProdotto, int
quantitaIniziale, int quantitaVenduta, int quantitaDisponibile)
```

```
{
    CodiceProdotto = codiceProdotto;
    NomeProdotto = nomeProdotto;
    QuantitaIniziale = quantitaIniziale;
    QuantitaVenduta = quantitaVenduta;
    QuantitaDisponibile = quantitaDisponibile;
}
}
```

Ottimo template per un esame! Strutturato bene, pulito e con la giusta separazione delle responsabilità.

Al tuo codice mancano esclusivamente le implementazioni dei metodi contrassegnati con `throw new NotImplementedException()`, oltre al collegamento logico di queste funzionalità all'interno dei menu interattivi (`GestisciMenuUtente` e `GestisciMenuAmministratore`).

Ecco la spiegazione passo passo di cosa manca e come implementarlo mantenendo intatte firme, parametri e tipi di ritorno.

1. Completamento delle Classi Dati e di Logica

Andiamo a sostituire i vari `NotImplementedException` blocco per blocco.

`ElementoCarrello`

Nel metodo `CambiaQuantitaScelta` dobbiamo validare che l'utente non inserisca cifre assurde o negative.

C#

```
public void CambiaQuantitaScelta(int nuovaQuantita)
{
    if (nuovaQuantita <= 0)
    {
        throw new ArgumentException("La quantità scelta deve essere maggiore di zero.");
    }

    QuantitaScelta = nuovaQuantita;
}
```

`CatalogoProdotti`

Qui gestiamo la persistenza in memoria del catalogo lato amministratore.

C#

```
public bool EliminaProdotto(string codiceProdotto)
{
    Prodotto? prodotto = CercaProdottoPerCodice(codiceProdotto);
    if (prodotto == null) return false;

    return prodotti.Remove(prodotto);
}

public bool ModificaPrezzoProdotto(string codiceProdotto, decimal nuovoPrezzo)
{
}
```

```

        Prodotto? prodotto = CercaProdottoPerCodice(codiceProdotto);
        if (prodotto == null) return false;

        try
        {
            prodotto.CambiaPrezzo(nuovoPrezzo);
            return true;
        }
        catch (ArgumentException)
        {
            return false;
        }
    }

    public bool ModificaQuantitaProdotto(string codiceProdotto, int
    variazioneQuantita)
    {
        Prodotto? prodotto = CercaProdottoPerCodice(codiceProdotto);
        if (prodotto == null) return false;

        try
        {
            prodotto.CambiaQuantita(variazioneQuantita);
            return true;
        }
        catch (Exception) // Cattura InvalidOperationException se va sotto zero
        {
            return false;
        }
    }
}

```

CarrelloUtente

Questo è il cuore della gestione del carrello prima dell'acquisto.

C#

```

public bool AggiungiAlCarrello(Prodotto prodotto, int quantita)
{
    if (quantita <= 0 || quantita > prodotto.QuantitaDisponibile) return false;

    ElementoCarrello? esistente = elementiCarrello.FirstOrDefault(e =>

```

```
        e.ProdottoSelezionato.CodiceProdotto.Equals(prodotto.CodiceProdotto,
StringComparison.OrdinalIgnoreCase));
```

```
    if (esistente != null)
```

```
    {
```

```
        int totaleRichiesto = esistente.QuantitaScelta + quantita;
```

```
        if (totaleRichiesto > prodotto.QuantitaDisponibile) return false;
```

```
        esistente.CambiaQuantitaScelta(totaleRichiesto);
```

```
    }
```

```
    else
```

```
    {
```

```
        elementiCarrello.Add(new ElementoCarrello(prodotto, quantita));
```

```
    }
```

```
    return true;
```

```
}
```

```
public bool ModificaQuantitaNelCarrello(string codiceProdotto, int
nuovaQuantita)
```

```
{
```

```
    if (nuovaQuantita <= 0) return false;
```

```
    ElementoCarrello? esistente = elementiCarrello.FirstOrDefault(e =>
```

```
        e.ProdottoSelezionato.CodiceProdotto.Equals(codiceProdotto,
```

```
StringComparison.OrdinalIgnoreCase));
```

```
    if (esistente == null || nuovaQuantita >
```

```
esistente.ProdottoSelezionato.QuantitaDisponibile) return false;
```

```
    esistente.CambiaQuantitaScelta(nuovaQuantita);
```

```
    return true;
```

```
}
```

```
public bool RimuoviDalCarrello(string codiceProdotto)
```

```
{
```

```
    ElementoCarrello? esistente = elementiCarrello.FirstOrDefault(e =>
```

```
        e.ProdottoSelezionato.CodiceProdotto.Equals(codiceProdotto,
```

```
StringComparison.OrdinalIgnoreCase));
```

```

        if (esistente == null) return false;

        return elementiCarrello.Remove(esistente);
    }

```

StoricoAcquisti

Filtriamo in modo sicuro per stringa senza fare caso a maiuscole/minuscole.

C#

```

public List<Acquisto> OttieniAcquistiPerUtente(string nomeUtente)
{
    return acquisti.Where(a => a.NomeUtente.Equals(nomeUtente,
StringComparison.OrdinalIgnoreCase)).ToList();
}

```

ServizioNegozio

Unisce il catalogo con il carrello e finalizza l'acquisto congelando lo stato del magazzino.

C#

```

public bool AggiungiProdottoAlCarrello(string codiceProdotto, int quantita)
{
    Prodotto? prodotto =
catalogoProdotti.CercaProdottoPerCodice(codiceProdotto);
    if (prodotto == null) return false;

    return carrelloUtente.AggiungiAlCarrello(prodotto, quantita);
}

public Acquisto ConfermaAcquisto(Utente utente)
{
    List<ElementoCarrello> elementi = carrelloUtente.OttieniElementi();
    if (elementi.Count == 0)
    {
        throw new InvalidOperationException("Impossibile confermare l'acquisto:
il carrello è vuoto.");
    }

    // Validazione preventiva del magazzino prima di toccare i dati
    foreach (var elem in elementi)
    {
        if (elem.QuantitaScelta > elem.ProdottoSelezionato.QuantitaDisponibile)

```

```

        {
            throw new InvalidOperationException($"Prodotto
{elem.ProdottoSelezionato.Nome} non sufficiente in magazzino.");
        }
    }

    List<ElementoAcquistato> acquistati = new List<ElementoAcquistato>();

    foreach (var elem in elementi)
    {
        // Scala dal magazzino passandogli il valore negativo
        elem.ProdottoSelezionato.CambiaQuantita(-elem.QuantitaScelta);

        acquistati.Add(new ElementoAcquistato(
            elem.ProdottoSelezionato.CodiceProdotto,
            elem.ProdottoSelezionato.Nome,
            elem.QuantitaScelta,
            elem.PrezzoUnitario
        ));
    }

    Acquisto nuovoAcquisto = new Acquisto(utente, acquistati);
    storicoAcquisti.RegistraAcquisto(nuovoAcquisto);
    carrelloUtente.SvuotaCarrello();

    return nuovoAcquisto;
}

```

2. Integrazione nei Menu dell'Applicazione

(**ApplicazioneNegozio**)

Ora dobbiamo sostituire i `Console.WriteLine("Funzionalità non implementata...")` presenti negli `switch` della classe principale con l'effettivo input dell'utente.

Sostituzioni nel Menu Utente (**GestisciMenuUtente**)

- **Caso "2" (Aggiungi prodotto):**

C#

```

case "2":
    Console.Write("Inserisci codice prodotto da aggiungere: ");
    string? codAdd = Console.ReadLine();

```

```

        int qtaAdd = LeggiInteroPositivo("Inserisci quantità: ");
        if (!string.IsNullOrEmpty(codAdd) &&
servizioNegozio.AggiungiProdottoAlCarrello(codAdd.Trim(), qtaAdd))
        {
            Console.WriteLine("Prodotto aggiunto al carrello!");
        }
        else
        {
            Console.WriteLine("Errore: prodotto non trovato o quantità non
disponibile/non valida.");
        }
        break;

```

- **Caso "4" (Modifica quantità):**

C#

```

case "4":
    Console.Write("Inserisci codice prodotto da modificare nel carrello: ");
    string? codMod = Console.ReadLine();
    int qtaMod = LeggiInteroPositivo("Inserisci nuova quantità: ");
    if (!string.IsNullOrEmpty(codMod) &&
carrelloUtente.ModificaQuantitaNelCarrello(codMod.Trim(), qtaMod))
    {
        Console.WriteLine("Quantità aggiornata!");
    }
    else
    {
        Console.WriteLine("Errore: prodotto non presente nel carrello o
disponibilità insufficiente.");
    }
    break;

```

- **Caso "5" (Rimuovi prodotto):**

C#

```

case "5":
    Console.Write("Inserisci codice prodotto da rimuovere dal carrello: ");
    string? codRem = Console.ReadLine();
    if (!string.IsNullOrEmpty(codRem) &&
carrelloUtente.RimuoviDalCarrello(codRem.Trim()))
    {
        Console.WriteLine("Prodotto rimosso dal carrello.");
    }
    else

```

```

{
    Console.WriteLine("Prodotto non trovato nel carrello.");
}

break;

```

- **Caso "7" (Conferma acquisto):**

C#

```

case "7":
    try
    {
        Acquisto acq = servizioNegozio.ConfermaAcquisto(utente);
        Console.WriteLine("\nAcquisto effettuato con successo!");
        servizioNegozio.StampaAcquisto(acq);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Errore durante l'acquisto: " + ex.Message);
    }

    break;

```

Sostituzioni nel Menu Amministratore (**GestisciMenuAmministratore**)

- **Caso "3" (Elimina dal catalogo):**

C#

```

case "3":
    Console.Write("Inserisci codice prodotto da eliminare dal catalogo: ");
    string? codDel = Console.ReadLine();
    if (!string.IsNullOrEmpty(codDel) &&
catalogoProdotti.EliminaProdotto(codDel.Trim()))
    {
        Console.WriteLine("Prodotto eliminato correttamente.");
    }
    else
    {
        Console.WriteLine("Impossibile trovare il prodotto specificato.");
    }

    break;

```

- **Caso "4" (Modifica Prezzo):**

C#

```

case "4":
    Console.Write("Inserisci codice prodotto: ");
    string? codPrice = Console.ReadLine();

```



```

        decimal nuovoPrezzo = LeggiPrezzoPositivo("Inserisci il nuovo prezzo:");
    };

    if (!string.IsNullOrEmpty(codPrice) &&
catalogoProdotti.ModificaPrezzoProdotto(codPrice.Trim(), nuovoPrezzo))
    {
        Console.WriteLine("Prezzo modificato con successo.");
    }
    else
    {
        Console.WriteLine("Errore: prodotto non trovato o prezzo non
valido.");
    }

    break;

```

- **Caso "5" (Modifica quantità magazzino):**

C#

```

case "5":

    Console.Write("Inserisci codice prodotto: ");
    string? codStock = Console.ReadLine();

    Console.Write("Inserisci variazione di magazzino (es. 5 per aggiungere,
-3 per prelevare): ");
    if (int.TryParse(Console.ReadLine(), out int variazione))
    {
        if (!string.IsNullOrEmpty(codStock) &&
catalogoProdotti.ModificaQuantitaProdotto(codStock.Trim(), variazione))
        {
            Console.WriteLine("Magazzino aggiornato con successo.");
        }
        else
        {
            Console.WriteLine("Errore: prodotto non trovato o l'operazione
porterebbe il magazzino sotto zero.");
        }
    }
    else
    {
        Console.WriteLine("Valore numerico non valido.");
    }

    break;

```

Incollando questi pezzi di codice al posto dei rispettivi blocchi `TODO`, la tua applicazione d'esame sarà completa al 100%, robusta contro i crash di input e pienamente funzionante nel rispetto dell'architettura iniziale.